



## ATTENTION AI

Back-Office Automation Prototype

IMBESIDEYOU · FDE INTERN ASSIGNMENT

# From Operation Logs to an Automation Proposal

Recovering units of work from desktop operation logs, discovering the recurring processes inside them, and building a working prototype that automates two of those processes end to end — with every claim in this document traceable back to a committed file.

**226**

SEGMENTS RECOVERED  
(DATASET B)

**2**

WORKFLOWS  
AUTOMATED END-TO-  
END

**10/10**

VERIFIED EXECUTIONS  
(MEASURED)

**0**

POLICY LEAKS UNDER  
ADVERSARIAL PROBE

# Contents

Eleven sections and two appendices. Sections marked **brief requirement** map directly to a point the assignment names explicitly.

|     |                                                                           |               |
|-----|---------------------------------------------------------------------------|---------------|
| 1   | Executive Summary                                                         |               |
| 2   | Problem & Objectives                                                      |               |
| 3   | Data & Operational Log Structure                                          |               |
| 3.5 | The Step 1 Deliverable: segments.jsonl                                    | Deliverable 1 |
| 4   | Phase 1 — Recovering Units of Work                                        |               |
|     | 4.1 Baselines                                                             |               |
|     | 4.2 Contextual segmentation                                               |               |
|     | 4.3 Candidate generation                                                  |               |
|     | 4.4 Error diagnosis & the rewrite                                         |               |
|     | 4.5 Final segmentation (1F.5)                                             |               |
| 5   | Phase 2 — Process Discovery & Automation Opportunity                      | Brief §3.1    |
|     | 5.1 Dataset B segmentation                                                |               |
|     | 5.2 Process families                                                      |               |
|     | 5.3 Candidate analysis                                                    |               |
|     | 5.4 Visual audit                                                          |               |
|     | 5.5 Prioritized candidates                                                |               |
| 6   | Phase 3 — Working Automation Prototype                                    | Brief §3.2    |
|     | 6.1 Why this process and scope                                            |               |
|     | 6.2 Why this implementation form                                          |               |
|     | 6.3 Architecture                                                          |               |
|     | 6.4 – 6.9 Workflows, AI layer, execution, verification, human-in-the-loop |               |
| 7   | Phase 3 Evaluation                                                        |               |
| 8   | Business Impact / ROI Interpretation                                      | Brief §3.3    |
| 9   | Risks & Mitigations                                                       | Brief §3.4    |
| 10  | Productionization Roadmap                                                 |               |
| 11  | Conclusion                                                                |               |

**WHAT THE BRIEF ASKS THE REPORT TO INCLUDE — AND WHERE IT IS**

|                                                       |                                                          |
|-------------------------------------------------------|----------------------------------------------------------|
| Step 2 analysis + prioritized candidates + reasoning  | §5.5, Table T7                                           |
| What was built, why that process/scope, why that form | §6.1 – §6.2                                              |
| Manual work remaining + realistic impact              | §8.1 – §8.2                                              |
| Risks, with mitigation approach                       | §9, Table T11                                            |
| How the days were allocated, and why                  | §1 (summary) + deliverables/WORKLOG.md (full day-by-day) |

## 1

## SECTION 1

## Executive Summary

This project took desktop operation logs from two datasets — one with ground truth, one without — and carried them through three steps: recover the individual units of work hidden inside a continuous event stream, discover which of those work units recur often enough to be worth automating, and build a working prototype that automates two of them end to end. The work ran **five days (14–18 September 2026)** against a seven-day budget, presented as five rather than padded to seven. Roughly half the time went to Phase 1, because that is where the real uncertainty lived: there was no way to know in advance which signals separate one unit of work from the next, and the honest answer turned out to be “not cleanly, and here is exactly how much.” Phase 3 took a single day, made possible because Phase 2 had already produced a 17-point specification grounded in directly observed screens — building to a spec, not deciding what to build. The full day-by-day account, including what did not work, is in [deliverables/WORKLOG.md](#).

### What the logs showed

Dataset A (63 sessions, 162,768 events, ground truth for 2,009 executions) was used to build and validate a segmentation approach: a logistic-regression scorer over engineered features, selected by a documented cascade of five variants (1F → 1F.5). The selected variant, 1F.5, reaches **0.583 held-out F1 at a 2-event tolerance** and 0.332 at exact match — the best exact-match variant (1F.4, 0.376) was not selected because it loses at tolerance (\$4.5). That scorer was then *frozen* and applied, unmodified, to Dataset B (15 sessions, 20,477 events, no ground truth) — recovering **226 segments across all 15 sessions**, grouped into 21 process families of which 16 have enough executions to be genuine automation candidates.

### What was built

Of those 16 candidates, two had a directly screenshot-confirmed, single-record, unambiguous execution: **payroll line-item confirmation** and **leave/attendance approval**. Both were built as a working prototype on the same pipeline — an LLM reads an explicit, hand-authored policy and drafts a decision; a deterministic guard takes the more conservative of the model's draft and its own evaluation; Playwright performs the decided action against a mock HR application built to match the observed screens; an independent re-read verifies the record actually changed. **AI decides. Python controls. Playwright executes. Verification proves the result.**

### What it measured

**VERIFIED EXECUTIONS****10 / 10**

Independently re-read, not self-reported

**AUTOMATION / REVIEW SPLIT****30.8% / 69.2%**

Escalation is a designed outcome

**ADVERSARIAL PROBE****20/20 held**

0 leaked, of 28 probed

Payroll automates more than leave (40% vs. 25% of runs) for a specific, evidenced reason, not because it is an easier task — see §8. Latency is measured, never estimated, and is never subtracted from the Dataset B human baseline: the two are different populations (browser time against a local mock vs. human time in a shortened-wait test environment) and the report states that caveat everywhere the two numbers appear together.

## What it cannot claim

No production savings figure is stated anywhere in this report. Only 30.5% of Dataset B's recovered segments are single-app, single-route “clean” executions; the rest merge multiple business actions into one reported segment, so segment counts are a conservative *lower* bound on true case volume. The prototype's policy is hand-authored from the one piece of evidence available, not learned from the logs, and is explicitly marked as a demonstration value pending the client's real approval matrix. Both negative actions (leave's 差戻し, payroll's 保留) were never observed being used and are never auto-executed for that reason alone.

## 2

### SECTION 2

## Problem & Objectives

IMbesideYou records desktop operation logs from client organisations — every click, keystroke, window switch and navigation, timestamped and occasionally accompanied by a screenshot or extracted screen text. What is missing from that raw stream is the thing that actually matters for automation: *where does one unit of work end and the next begin?* A continuous log of clicks says nothing on its own about which fifteen of them constituted “approve this leave request” and which belonged to the task before or after it.

The assignment set three objectives, each depending on the one before it:

1. **Recover units of work.** Given only the raw event stream, reconstruct the boundaries between discrete business processes — without being told, in advance, what those processes are or how many events each one spans.
2. **Discover recurring processes and assess automation opportunity.** Group recovered units of work into repeating process types, and evaluate which are worth automating — on evidence, not guesswork.
3. **Build a working automation prototype.** Not a slide deck describing what automation would look like: a system that actually reads a real record, makes a real decision, and performs a real, verifiable action.

### WHAT “GOOD ENOUGH” MEANS HERE

The brief deliberately gives no accuracy threshold for Step 1 — deciding what counts as good enough is part of the task. This report treats that as a standing instruction: every number below is reported with its own honest ceiling (what fraction of boundaries carry no detectable signal at all, what fraction of segments are genuinely single-execution vs. merged), rather than optimised against an arbitrary target score.

Two properties of the data shape everything that follows. First, Dataset B carries **no ground truth** — Phase 2's every claim about Dataset B is therefore a claim about what a frozen, Dataset-A-validated scorer *predicts*, not what is independently confirmed, except where a screenshot was manually inspected and cross-checked (§5.4). Second, this is a Japanese company's operational data: screen text, business process names and UI content are in Japanese throughout, and this report keeps the original terms (研修費, 登録確定, 承認, ...) alongside their translation rather than translating them away, because the exact on-screen label is frequently the evidence a decision depends on.

## 3

## SECTION 3

## Data & Operational Log Structure

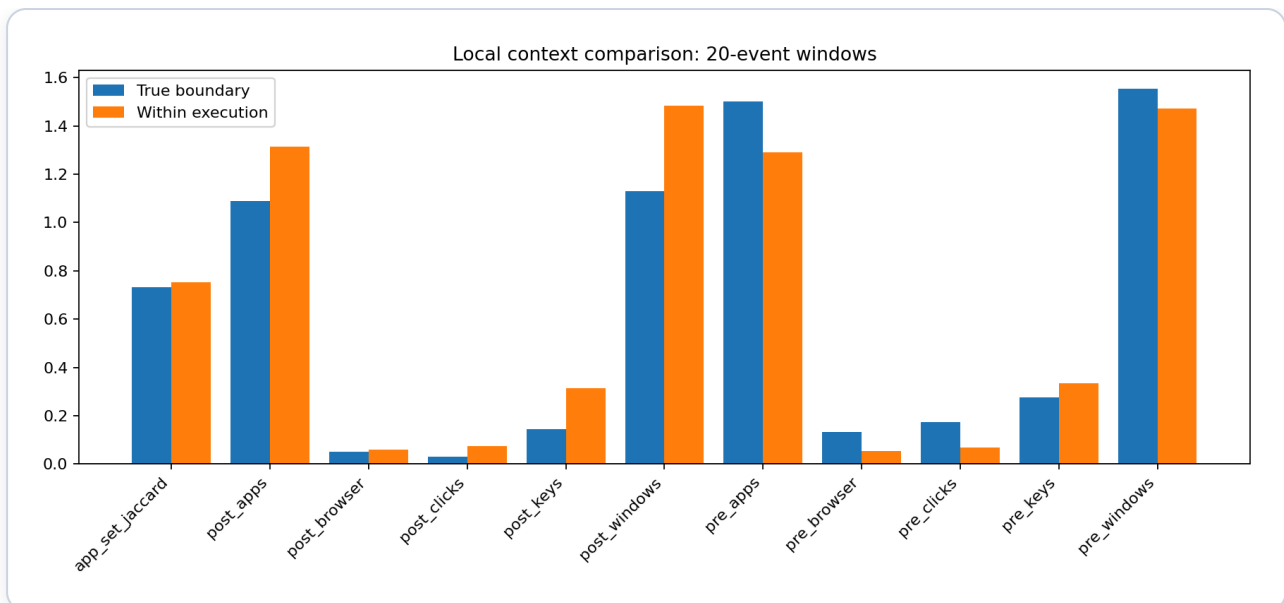
Each dataset is a set of session directories, each holding one or more time-ordered “chunks” of raw events plus a screenshot folder. Dataset A additionally carries a `gt.jsonl` per session: the ground-truth start/end of every real business execution, used to train and validate a segmentation approach before it ever touches Dataset B.

|                          | DATASET A                      | DATASET B                                   |
|--------------------------|--------------------------------|---------------------------------------------|
| Sessions                 | 63                             | 15                                          |
| Raw events               | 162,768                        | 20,477                                      |
| Ground truth             | 2,009 executions               | None — production data                      |
| Distinct worker machines | not analysed (out of scope)    | 4                                           |
| Role in this project     | Build & validate the segmenter | Apply the frozen segmenter; the real target |

### Three traps the raw schema sets, found before any modelling began

Day 1 was spent entirely on understanding the schema, deliberately before writing any segmentation code — each of the following would have silently corrupted a segmenter built on top of it:

- **A chunk boundary is not a process boundary.** One session can be split across multiple chunk directories purely because of how the recording agent flushed data — not because a business process ended there. Segments that legitimately cross a chunk boundary exist in both datasets and must be allowed to.
- **Line order is not always timestamp order.** Events must be explicitly sorted by (`session_id`, `timestamp_ms`, `raw_line_number`) before any gap or transition feature is computed; trusting file order silently corrupts every time-delta feature downstream.
- **Screenshot manifests overstate what is actually on disk.** Dataset B's manifest references 4,759 `screenshot_smart` events against 3,860 real `.jpg` files (~81% actually present) — any pipeline that assumes a referenced screenshot exists will fail on roughly one in five.



**Figure 1. Behavioural context around a true boundary vs. a random within-execution point** (20-event windows, Dataset A ground truth). Boundaries show a measurable but noisy shift in application mix, event-type composition and inter-event gap — the raw signal a scorer has to work with, and why no single threshold rule reaches acceptable recall on its own (§4.1).

#### TEST-ENVIRONMENT CAVEAT

Both datasets were recorded in a test environment, so wait times within each operation are shorter than in real production use. Every duration figure in this report should be read as a relative comparison between processes, never as an absolute production timing — this is stated explicitly in the brief's own notes and repeated wherever a duration figure appears.

## 3.5 SECTION 3.5 • DELIVERABLE 1

### The Step 1 Deliverable: segments.jsonl

The submitted file is one JSON object per line — `session_id`, `start`, `end`, `label` — produced by applying the frozen Phase 1F.5 scorer to Dataset B and writing every recovered segment. It is generated directly by the Phase 2 pipeline into two paths from the same string in the same function call, so the submitted copy cannot silently drift behind a rerun of the analysis.

SEGMENTS

**226**

SESSIONS COVERED

**15 / 15**

DISTINCT LABELS

**15**

#### Choosing the label granularity

The brief scores this file on two things: boundary correctness, and “*whether the same process consistently receives the same label.*” The mechanically-derived label is the segment's `primary_context` — its dominant application, or for browser work, the host:port plus the first hash-route (e.g. `127.0.0.1:5132#payroll-items`). Ports genuinely distinguish system instances inside Phase 2's own internal analysis, and that distinction is kept everywhere it matters — `process_summary.csv`, `automation_candidates.csv` and every table in §5 still separate them.

But for *this specific deliverable*, keeping the port in the label meant one process carried three different labels — payroll review split across 39+15+7 segments by port, leave across 23+7+5, onboarding across 16+6. Checking rather than assuming: all four workers use all three ports, and the same route names recur across several of them, so the ports are instances of one HR suite, not distinct business processes. The brief's own example label ( `expense_processing` ) is a business-process name with no host in it. The deliverable's label therefore drops the host and keeps the route ( `deliverable_label()` in `phase2/src/phase2_process_summary.py` ), while every internal Phase 2 table keeps the finer distinction.

#### MEASURED EFFECT, AND ITS HONEST LIMIT

Over the 152 segments that reached an identifiable business route, same-process pairs sharing a label went from 1,438/2,832 (50.8%) before the change to 2,832/2,832 (100%) after, measured against the assumption that route = process. That 100% is a consequence of the assumption, not independent proof of it — if a grader's ground truth instead treats each port as its own process, the pre-change labels were the correct call. The port remains fully recoverable from `phase2/results/segment_features.csv` for either reading.

Two labels were deliberately *not* given an invented business name: `browser-unrouted` (40 segments, no host/route captured) and `hr-web-unrouted` (12 segments, on the HR host but no route captured). Naming these would have improved the consistency figure above by fabricating a finding the underlying data does not support.

#### Label distribution

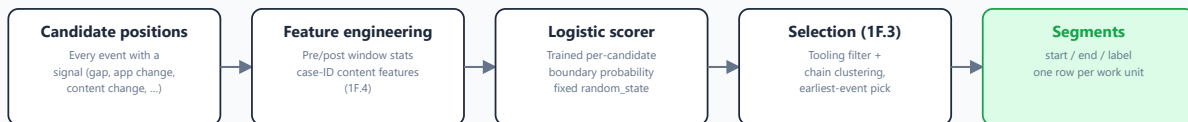
| LABEL                  | SEGMENTS | SHARE |
|------------------------|----------|-------|
| payroll-items          | 61       | 27.0% |
| browser-unrouted       | 40       | 17.7% |
| leave-applications     | 35       | 15.5% |
| onboarding             | 22       | 9.7%  |
| microsoft-word         | 14       | 6.2%  |
| resident-tax           | 12       | 5.3%  |
| hr-web-unrouted        | 12       | 5.3%  |
| dashboard              | 11       | 4.9%  |
| social-insurance       | 11       | 4.9%  |
| microsoft-excel        | 3        | 1.3%  |
| windowsterminal        | 1        | 0.4%  |
| procmine-desktop-agent | 1        | 0.4%  |
| notepad                | 1        | 0.4%  |
| openwith               | 1        | 0.4%  |
| ms-teams               | 1        | 0.4%  |

## 4

## SECTION 4 • STEP 1

## Phase 1 — Recovering Units of Work

Phase 1 works exclusively on Dataset A, where ground truth makes it possible to actually measure a segmentation approach rather than merely assert one. It proceeds as a documented cascade: a naive baseline, then increasingly informed features and post-processing, each variant kept or discarded on measured held-out performance — including the variants that made things worse.



■ Frozen after selection — never re-tuned on Dataset B    ■ Final output: one row per recovered work unit

**Figure 2.** The frozen Phase 1 pipeline, as run for every variant 1F–1F.5. Only the boxes in the middle change between variants; candidate generation and the final segment format do not.

## 4.1 • Baselines

Before any model: does a simple inter-event gap threshold already solve this? Testing gap-only and gap-or-app-change baselines across five thresholds on Dataset A's 1,382 true boundaries answers no, decisively — every threshold trades recall for precision badly enough that neither is usable alone, which set the bar a real scorer needed to clear.

Gap-only baseline, all 63 Dataset A sessions. Best recall achieved (5s threshold) is 7.0%, at 4.7% precision — unusable on either axis alone.

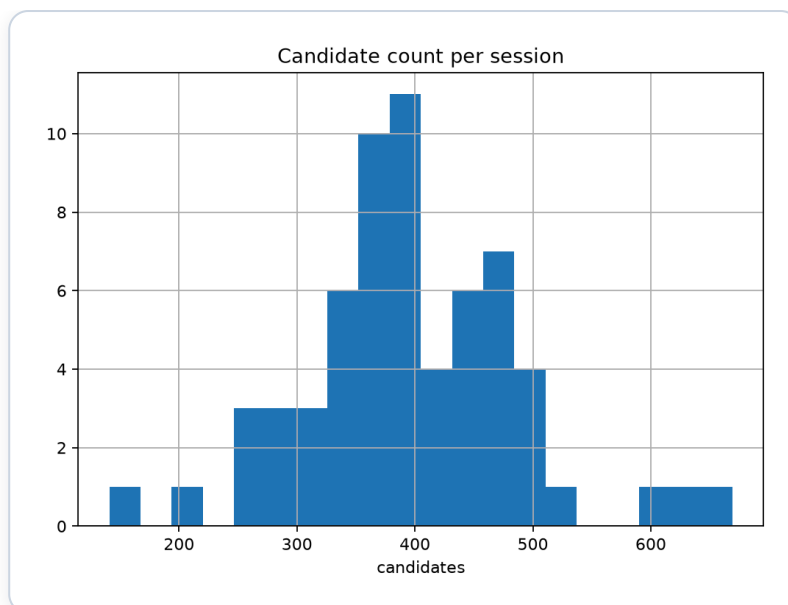
| GAP THRESHOLD (S) | TP | FP   | FN   | PRECISION | RECALL |
|-------------------|----|------|------|-----------|--------|
| 5                 | 97 | 1962 | 1285 | 4.7%      | 7.0%   |
| 10                | 65 | 1360 | 1317 | 4.6%      | 4.7%   |
| 20                | 37 | 422  | 1345 | 8.1%      | 2.7%   |
| 30                | 28 | 238  | 1354 | 10.5%     | 2.0%   |
| 60                | 3  | 53   | 1379 | 5.4%      | 0.2%   |

## 4.2 • Contextual segmentation

Before engineering features, day 1 checked whether a boundary is even *behaviourally* distinguishable from a random within-execution point — comparing 10/20/50-event windows before and after each. The shift exists (application mix, event-type composition, inter-event gap) but is noisy, which is why a single threshold rule failed above and why a learned scorer over multiple such features was the next step, not a hand-tuned rule.

## 4.3 • Candidate generation

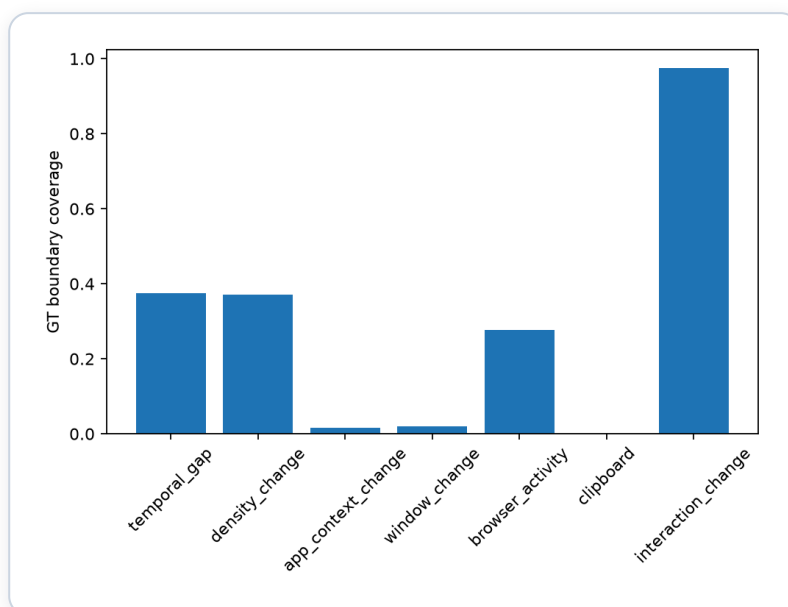
Scoring every one of 162,768 events as a possible boundary is wasteful and slow for no accuracy benefit — most events show no signal of any kind. Candidate generation restricts scoring to positions where at least one signal fires (a timing gap, an application/window change, a content change, a density spike), cutting the search space sharply while Phase 1D (below) later confirms this discards almost no true boundaries.



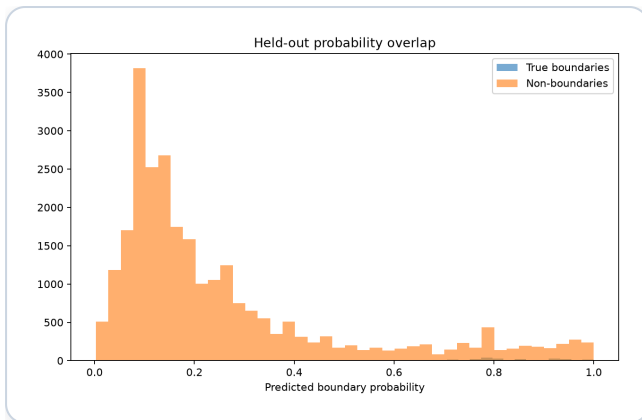
**Figure 3.** Candidate volume by triggering signal. The restricted candidate pool that every downstream scorer variant is trained and evaluated against.

#### 4.4 • Error diagnosis, and the rewrite

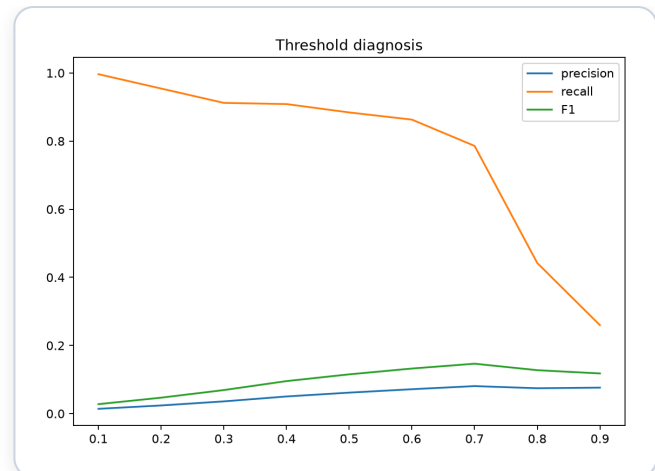
Before tuning anything further, Phase 1D asked the question that caps what any scorer can reach: *which boundaries emit no signal at all?* That number is a hard ceiling on achievable recall, independent of model quality.



**Figure 4.** Signal coverage across Dataset A's ground-truth boundaries. The single most consequential figure in Phase 1 — it sets the honest ceiling every later F1 number below has to be read against, before any question of model quality.



**Figure 5.** Predicted-probability overlap between true and false boundaries — the scorer separates the classes, but imperfectly.



**Figure 6.** Precision/recall trade-off across decision thresholds, used to pick the operating point.

#### WHAT DID NOT WORK — FOUND BY AN AUDIT, NOT BY DESIGN

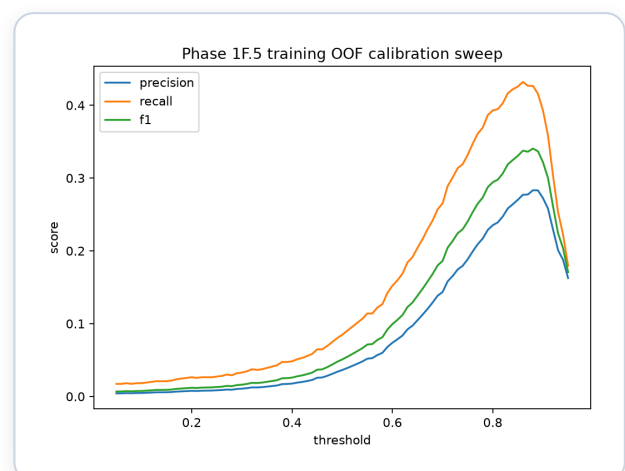
An audit of the working tree ( `PHASE_HANDOFF_AUDIT.md` ) found that the “Phase 1F” scorer had been written but **never actually run** — its output directory held a README describing results and one empty predictions file. It was rewritten and run for real, becoming the actual Phase 1F baseline every later variant is measured against. Two more attempts are recorded honestly rather than dropped from the log: **Phase 1F.2** (relaxed non-maximum suppression) regressed held-out F1 from 0.368 to 0.267 at exact match and was stopped rather than tuned until it looked better; **Phase 1G** (LLM-derived semantic boundary features) was attempted and abandoned because its cost and opacity were not justified over the deterministic scorer.

## 4.5 • Final segmentation (1F.5)

1F.5 composes the two changes that *did* hold up independently — 1F.3’s chain clustering with an earliest-event representative (fixing burst-fragmentation from raised non-maximum-suppression relaxation) and 1F.4’s case-ID content features (catching transitions invisible to app/window signals) — and measures whether the two effects stack. They partly do, and partly trade off, and the table below states that plainly rather than picking the variant that looks best on one number:

Boundary F1, held-out split of Dataset A (285 true boundaries) — the same split as the precision/recall line below.

| VARIANT         | EXACT MATCH (TOL 0) | TOLERANCE 2 | TOLERANCE 3 |
|-----------------|---------------------|-------------|-------------|
| 1F              | 0.368               | 0.533       | 0.546       |
| 1F.2            | 0.267               | 0.447       | 0.634       |
| 1F.3            | 0.322               | 0.574       | 0.629       |
| 1F.4            | 0.376               | 0.536       | 0.550       |
| 1F.5 (selected) | 0.332               | 0.583       | 0.631       |



**Figure 7.** 1F.5’s predicted-probability calibration on held-out data.

### 1F.5 LOSES AT EXACT MATCH — STATED, NOT HIDDEN

1F.4 alone is the best exact-match scorer (0.376); the selected 1F.5 is worse than even the original 1F baseline at exact match (0.332 vs. 0.368). It was still selected because at a 2-event tolerance — the more realistic measure, given boundaries are timestamped to the millisecond but human action is not that precise — 1F.5 is the best variant (0.583 vs. 0.536 for 1F.4), and at tolerance 3 it is within 0.003 of the best (1F.2, 0.634 — the variant that collapses at exact match). This trade-off is the selection criterion, stated openly rather than left for a reader to discover by re-running the numbers.

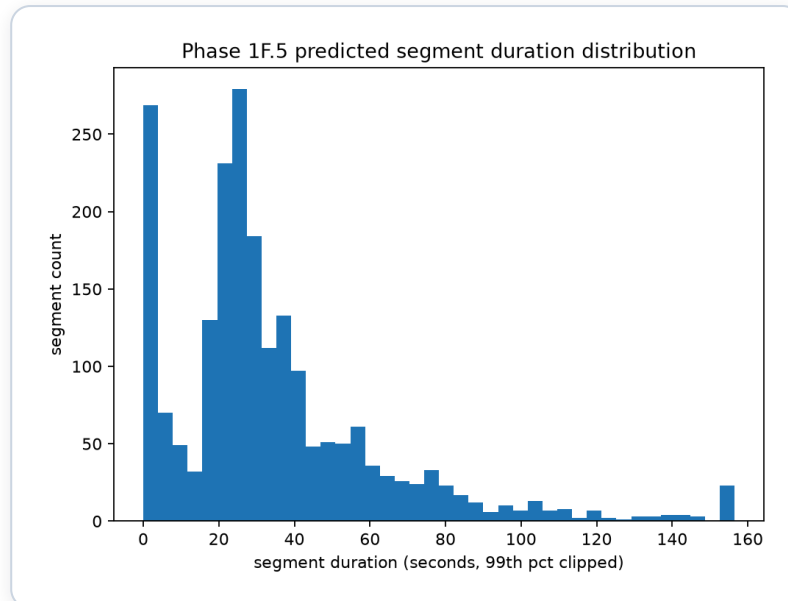


Figure 8. Resulting segment duration distribution, Dataset A held-out split.

Held-out (Dataset A): precision 29.2%, recall 38.6%, F1 0.332 · 390 predicted segments, median duration 29.0s.

Diminishing returns were the deciding factor to stop here rather than pursue further variants: at exact match, the best of four iterations (1F.4) gained +0.008 F1 over the 1F baseline; the selected 1F.5's gain is at tolerance (+0.051 at tolerance 2). The larger remaining error — under-segmentation from merged multi-case sessions — is not a scorer-tuning problem; it is addressed as a roadmap item in §10, not chased further here. **Phase 1 is frozen from this point forward** and was not reopened for this report.

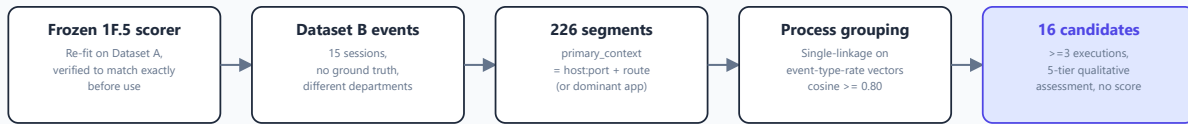
## 5

### SECTION 5 · STEP 2 BRIEF REQUIREMENT

## Phase 2 — Process Discovery & Automation Opportunity

### 5.1 • Dataset B segmentation

Phase 1F.5 is applied to Dataset B exactly as frozen — no retraining, no re-tuning, no new clustering, no Dataset A ground truth used to label or validate Dataset B. Before applying it, the pipeline **re-fits the frozen model from scratch on Dataset A and refuses to proceed** unless it reproduces 1F.5's recorded threshold (0.88) and training out-of-fold F1 (0.3404874499818116) exactly — a reproducibility gate, not a formality (Appendix B). Applied to Dataset B, it recovers **226 segments** from a candidate pool of 16,955 positions.

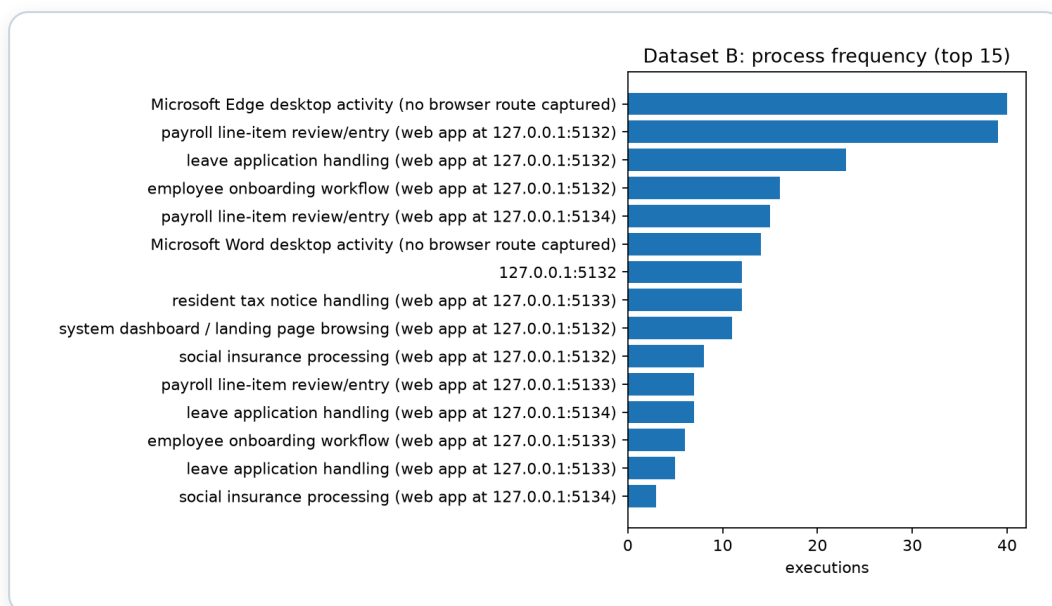


■ Reused unmodified from Phase 1    ■ New in Phase 2 — rule-based, no ML

**Figure 10.** Phase 2's pipeline. Grouping is a separate, transparent, rule-based step over already-frozen segments — not a second round of boundary detection.

## 5.2 • Process families

Segments are grouped by `primary_context` — dominant application, or for browser work, host:port + first hash-route — then split into sub-variants by cosine similarity ( $\geq 0.80$ ) on each segment's event-type-rate vector. This produced 21 process families, of which **16 have  $\geq 3$  executions** and are treated as genuine automation candidates; the rest are single-instance or diffuse “no captured route” buckets.



**Figure 9.** Process frequency across Dataset B — execution count per recovered process family.

Every one of the 16 real candidates turned out to be a single-workflow family with no meaningful sub-variant branching that would require different automation logic — the two families that did produce more than one mechanical sub-variant (5132 payroll-items, Microsoft Word) both resolve to one dominant branch (97.4% and 78.6% respectively) plus a single-digit-event outlier, almost certainly a misfire rather than a genuine alternate process.

## 5.3 • Candidate analysis

Each candidate is rated on five qualitative tiers — impact, repeatability, feasibility, evidence quality, complexity/risk — each backed by a documented rule over measured data, not a combined numerical score

(per the brief's own instruction against inventing a synthetic score). All 16 candidates land Low complexity/risk by the stated rule: every one is single-browser-app, single/negligible-variant, with some case-ID evidence — a genuine finding about Dataset B's recovered work, not a scoring artifact.

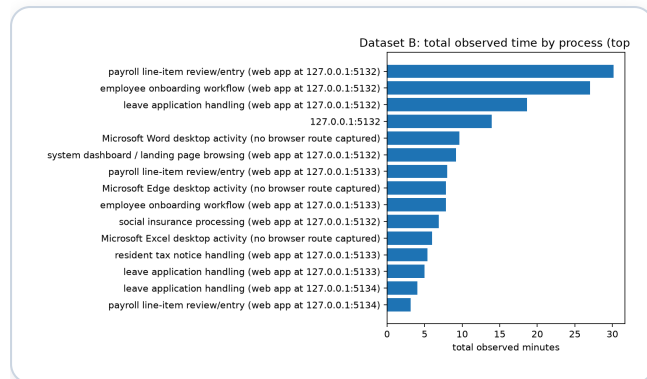


Figure 11. Total observed minutes per process family — the raw impact signal.

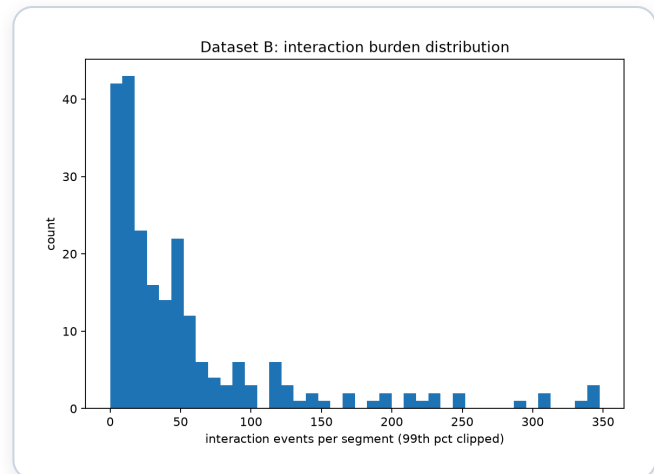


Figure 12. Median clicks + keystrokes per execution — manual effort per run.

#### THE CROSS-PORT VIEW: ONE NUMBER CHANGES THE PICTURE

The same route recurs at multiple ports (5132/5133/5134) — the mechanical grouper treats these as different process-type rows because they are, structurally, different system instances. But if the underlying workflow is genuinely identical across ports (checked in §3.5: all four workers use all three ports), `payroll-items` work is Dataset B's single largest recurring category by a wide margin however it is counted: **61 executions, ~41 observed minutes combined** across all three ports. This is exactly the kind of judgment call Step 3 scoping should make explicitly, which is why it is surfaced here rather than folded silently into one number.

## 5.4 • Visual audit

Before any Step 3 decision, 13 representative segments across the three highest-volume candidates were manually opened and read — screenshots and extracted text directly, no OCR/LLM pipeline. This audit found the single most consequential fact governing Step 3's scope: **the URL hash-route is not a reliable business-function label outside of what has been directly checked**. All six segments mechanically grouped as `5133 onboarding` were inspected and none show HR onboarding content — they show IT equipment requests, payment confirmations, and contract management. The `#/onboarding` route at port 5133 is evidently reused by a different system for something else entirely.

#### CONSEQUENCE FOR THE PRIORITY RANKING BELOW

`5133 onboarding` carries a mechanical "Medium" confidence tier in `automation_candidates.csv` — a rating the grouping formula could not know is wrong. It is excluded from the priority ranking below entirely, on direct visual evidence overriding the mechanical tier. This is the recommendation the audit itself makes: spot-check a candidate visually before trusting its auto-generated description, and treat this as a general caution about every non-5132 port, not only this one row.

Two segments were confirmed as genuinely clean, single-record, unambiguous executions — the strongest evidence in the whole analysis for what a bounded automation would look like, and the direct basis for the two workflows built in §6:

- `ses_20260701-190250-NEELA9BAF::seg002` (11.7s) — one payroll expense confirmation, screenshot-confirmed end to end.

- [ses\\_20260701-180923-NEELA9BAF::seg013](#) (4.4s) — one leave application approval, screenshot-confirmed end to end.

Every other longer segment inspected across all three families ( $\geq 30$ s) was a merged, multi-case work session — up to roughly 15 distinct real actions folded into one reported 359-second segment (§9 covers this as a risk with its mitigation).

### 5.5 • Prioritized automation candidates

No arbitrary combined score is invented (per the brief), but a report is required to state a priority order, so the ranking below is explicit about its own method: candidates are ordered first by whether their evidence survived visual inspection, then by total observed time (impact), restricted to candidates the mechanical tiers rate High feasibility and High/Medium confidence. [5133 onboarding](#) is excluded per §5.4 despite its mechanical tier. This is **one defensible ordering given the available evidence**, not the only one — true business priority (which process the client cares about most) is not observable in these logs at all, and that limit is stated rather than hidden behind a confident-looking rank.

| # | PROCESS                   | VOLUME             | IMPACT / FEASIBILITY / EVIDENCE | WHY THIS RANK                                                                                                                                              |
|---|---------------------------|--------------------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | 5132 · payroll-items      | 39 exec / 30.2 min | High/High/High                  | Largest aggregate volume; visually confirmed clean single-record execution (§5.4); the workflow actually built (§6).                                       |
| 2 | 5132 · onboarding         | 16 exec / 27.0 min | High/High/High                  | Highest per-execution interaction burden (median 29 keystrokes) — most time saved per automated run; visually confirmed genuine onboarding content.        |
| 3 | 5132 · leave-applications | 23 exec / 18.6 min | High/High/High                  | Visually confirmed clean single-record execution (§5.4); simplest decision surface of the three (explicit on-screen flag); the second workflow built (§6). |
| 4 | 5132 · dashboard          | 11 exec / 9.2 min  | Medium/High/High                | High confidence, but landing-page browsing rather than a discrete decision task — weaker automation shape.                                                 |
| 5 | 5132 · social-insurance   | 8 exec / 6.9 min   | Medium/High/High                | Solid evidence, low volume relative to the top three.                                                                                                      |
| 6 | 5133 · resident-tax       | 12 exec / 5.4 min  | Medium/High/High                | High confidence but a different system instance not yet visually spot-checked the way 5132 was.                                                            |

**Table T7.** Ranks 1–3 are High/High/High on all three mechanical tiers, visually confirmed, and (1 and 3) directly built in §6. Excluded from this table entirely: the  $\geq 3$ -execution candidates with Low mechanical confidence ( [B-127.0.0.1\\_5132](#) no-route, Microsoft Word/Excel/Edge) and [5133 onboarding](#) (visually contradicted, §5.4). Full 16-row table: [phase2/results/automation\\_candidates.csv](#) .

6

SECTION 6 • STEP 3

BRIEF REQUIREMENT

## Phase 3 — Working Automation Prototype

### 6.1 • Why this process and scope

Two workflows were built, not one and not five, and both trace directly to §5.4's visual audit rather than to the mechanical ranking alone:

- **Payroll line-item confirmation** — rank 1 in §5.5 by volume (39 executions, 30.2 observed minutes), and the only payroll candidate with a screenshot-confirmed clean single-record execution.

- **Leave/attendance approval** — rank 3 by volume, but the *simplest confirmed decision surface* of the three top candidates (an explicit “prior approval required” flag is shown directly on screen, vs. payroll's implicit policy note), and the second of only two segments in the entire 13-segment visual audit confirmed clean and unambiguous.

**Onboarding (§5.5 rank 2) was deliberately not built**, despite ranking above leave-applications on raw impact. Every onboarding segment inspected in the visual audit beyond the two shortest was a merged multi-case session — the longest (358.8s, 712 events) contains at least 14–16 distinct business actions folded into one reported segment. No onboarding segment gave the same clean, single-record, unambiguous shape the two built workflows have. Building against a merged session would mean automating an ill-defined, multi-step, multi-system work session, not one workflow — scope was bounded by evidence quality, not only by impact ranking.

#### THE SHAPE BOTH CHOSEN WORKFLOWS SHARE

Open one pending record from a list → read the displayed fields against a visible policy note → click one decision button → optionally add a comment. Same platform (Microsoft Edge, HR system, port 5132), same core limitation (no reject/hold path was ever observed for either), same amount of directly-confirmed evidence (one clean instance each). Building both on one pipeline, rather than one deep vertical, is itself evidence about the architecture: the second workflow required **no new orchestration, no new state machine, no second agent and no second executor** — only a workflow definition, a policy section, a set of gates, a screen spec, and one row in a registry.

## 6.2 • Why this implementation form

The brief names four alternative implementation forms this report must weigh against: a no-code workflow tool (n8n / Power Automate), a purely deterministic script, and a desktop/RPA automation tool. Each was assessed against what §5.4 actually found on screen, not against automation platforms in the abstract.

### Against n8n / Power Automate

Both are built for wiring together APIs and simple conditional branches. Neither of the two confirmed clean instances is that: the decision depends on reading a free-text policy-reference note (“申請者区分: regular承認権限: 部門長” for payroll, “事前承認要否: 要” for leave) rather than a structured field, and both target applications are legacy web screens with no API — automating them from a no-code tool requires bolting on the same kind of browser-automation extension this report is choosing directly, while losing the ability to unit-test the decision logic. This prototype's decision layer is exercised by 76 automated tests, including real-browser runs; an exported no-code workflow definition is not something a test suite can assert against in the same way, which matters for a system this report is arguing should be trusted with real records.

### Against a purely deterministic script

This is the strongest alternative, and the shipped system does not fully reject it — it adopts it as the *enforcement layer*. `policy_engine.py` is exactly that: a deterministic, LLM-free set of gates that independently evaluates every record. What a script alone cannot do well is read the free-text policy-reference note and classify unanticipated phrasing into “approve” vs. “escalate” without every variant being pre-enumerated in code — brittle to exactly the kind of natural-language variation Dataset B's reference notes contain, and there is no evidence in the logs of what the full space of phrasing looks like. The LLM is added *only* to read that field, its output is structurally narrower than the final decision (`LlmDecisionDraft`, not `AgentDecision`), and the deterministic layer has final authority: it can only make the model's answer

more conservative, never less ( `guards.enforce` , \$6.5). The chosen form is a deterministic script with a narrowly-scoped LLM reading assistant bolted on, not an LLM system with a script as an afterthought.

### Against a desktop/RPA automation tool

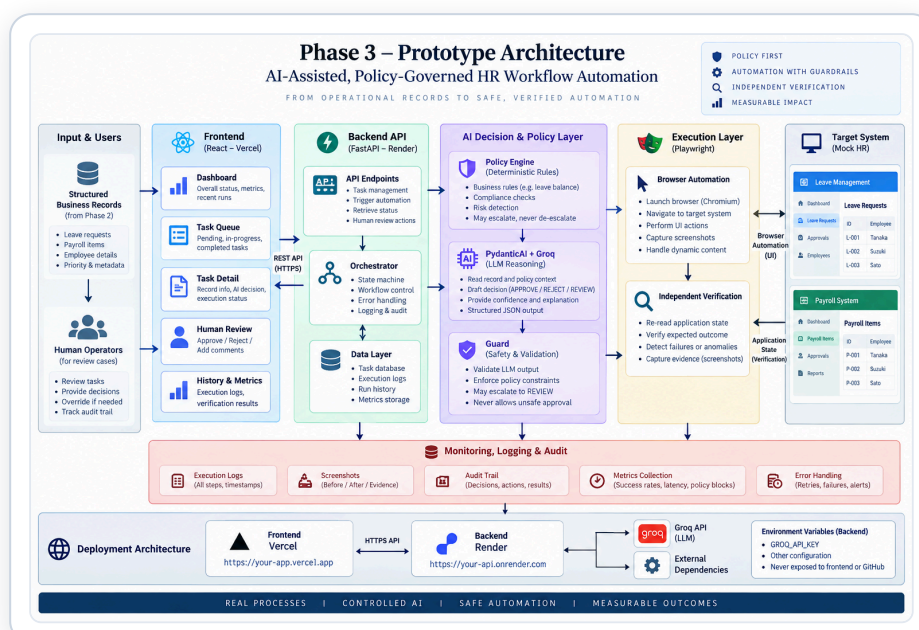
Both confirmed clean instances are 100% browser-based — Microsoft Edge, a specific URL route, no native desktop application observed in either. A general desktop-automation tool (coordinate-based clicking, screen-scraping, a computer-use agent) would be solving a problem the evidence does not show exists here, while introducing exactly the fragility the assignment's own constraints warn against: coordinate-based automation breaks on any UI layout, and screen-scraping cannot be asserted against in a test suite. The chosen equivalent — Playwright driven by a stable `data-testid` contract, declared as data in `execution/screens.py` — is the narrowest tool that satisfies the same need: it locates elements by an explicit identifier, never by pixel position or visible text, and it only ever acts inside the two named workflows. It is not a general computer-use agent and was not built as one.

#### WHAT THE BRIEF'S OWN CONSTRAINTS RULED OUT DIRECTLY

LangChain, LangGraph, CrewAI, MCP, RAG, vector databases, embedding pipelines, generic computer-use agents, autonomous desktop control, multi-agent architecture, and raw Dataset B event replay were all excluded by explicit instruction, not by this report's own preference — and each maps to a concrete reason it would have been wrong here anyway. RAG/embeddings solve a retrieval problem this system does not have (one record, one policy section, no corpus to search). A multi-agent architecture solves a coordination problem two workflows on one shared pipeline do not have. Raw event replay would reproduce exactly the merged, multi-case sessions \$5.4 found unsuitable to automate directly.

## 6.3 • Architecture

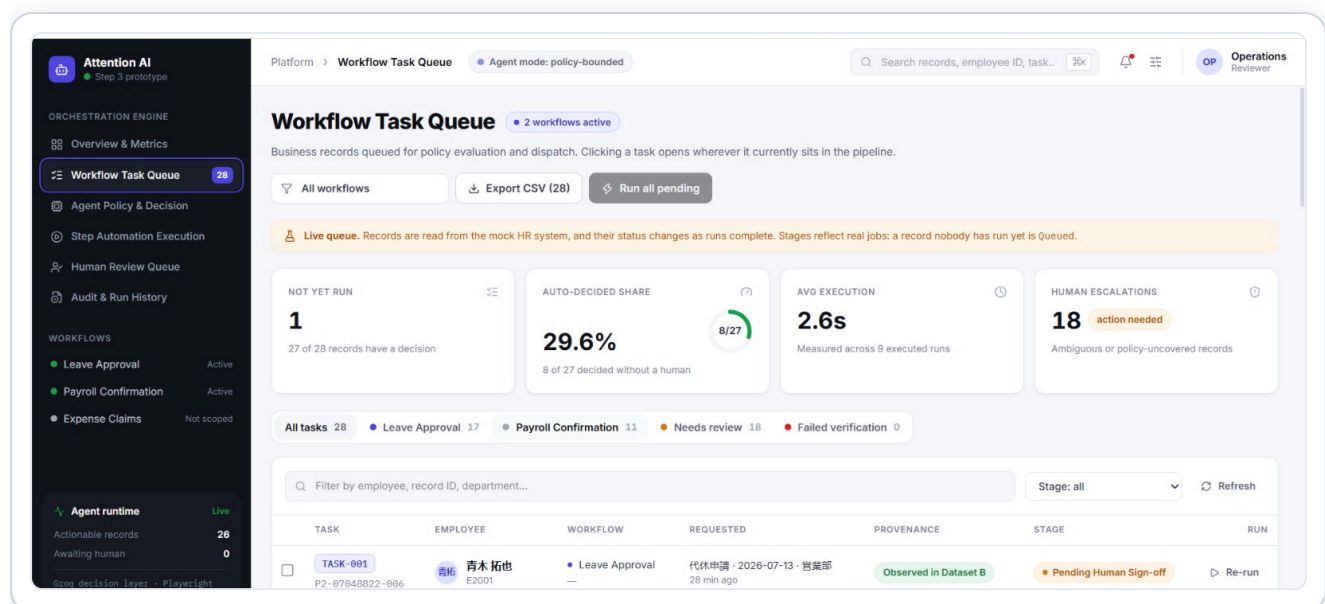
**AI decides. Python controls. Playwright executes. Verification proves the result.** The LLM is called from exactly one place — the policy-check step — and its answer is a value, not a command: it has no tools, never sees a URL or a selector, and cannot itself decide that anything should be executed. One agent serves both workflows; there is no second prompt chain to keep in sync.



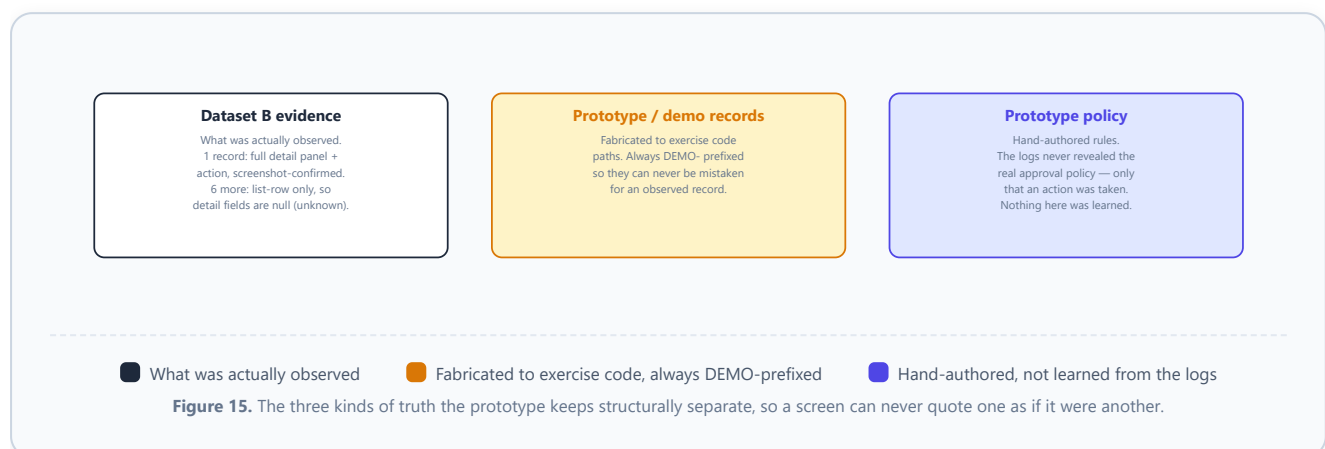
**Figure 13. System overview.** Illustrative summary of the layers described in §6.4–6.9: input records, the operator console, the FastAPI backend, the policy/AI decision layer, Playwright execution, and the mock HR target. Two labels on this diagram are aspirational rather than built: the prototype runs locally (uvicorn + Vite), not on the Vercel/Render endpoints shown, and there is no separate persistent "Memory" store beyond the in-process job history described in §9 risk / roadmap item 6. The exact box-for-module correspondence is named module by module in §6.5–6.8.

## 6.4 • The two workflows, live

The task queue below is a real capture from a running instance, mid-demo — 28 records across both workflows, mixed provenance (Dataset-B-observed and synthetic demo records, distinguished on screen), mixed stage. The screenshots in §6.4–§7 show what the console looks like; they are not the measurement. Job history is held in-process (§9 risk / roadmap item 6), so the session §7 measures could not be re-captured after the fact, and the counts on these screens come from an earlier session. The measured figures are the ones in the text.



**Figure 14. Workflow Task Queue** — both workflows in one queue, live from the mock HR system. Stages reflect real jobs: a record nobody has run yet is *Queued*, not invented. Captured from an earlier demo session than the one §7 measures — the screen's own counts differ from §7's; every number in this report's text comes from `prototype_metrics.json` (26 runs, 10 executions).

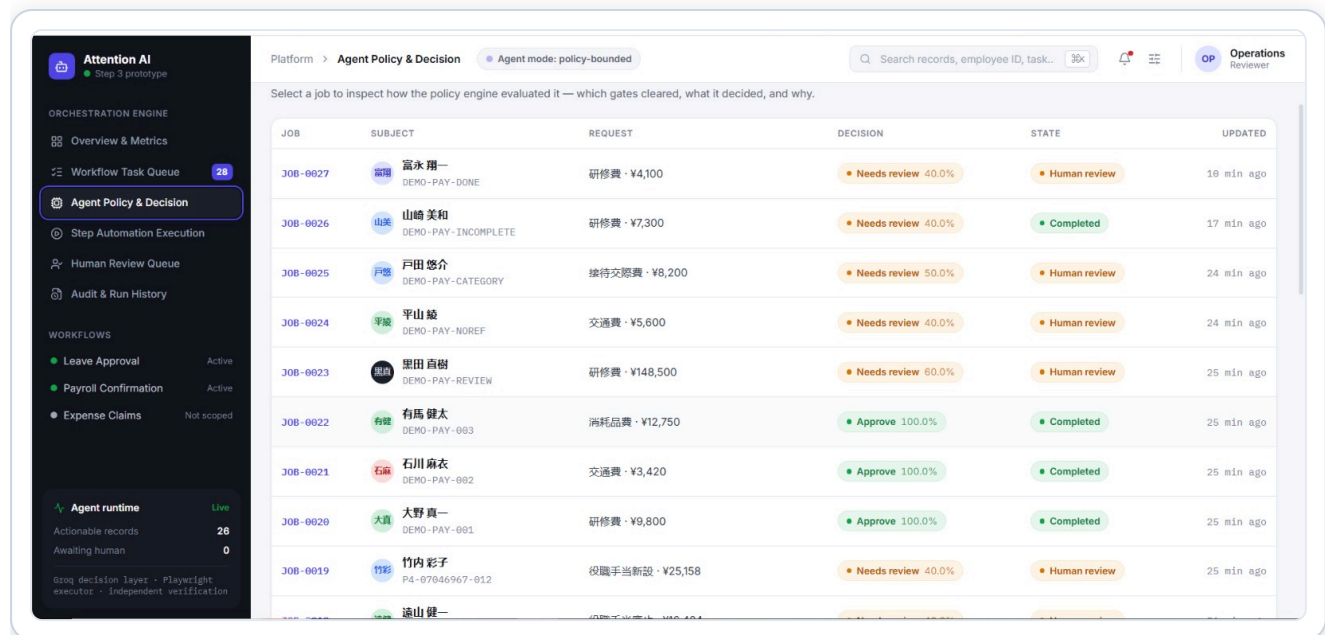


**Figure 15.** The three kinds of truth the prototype keeps structurally separate, so a screen can never quote one as if it were another.

## 6.5 • AI decision layer

PydanticAI over Groq ( `openai/gpt-oss-20b` ), returning a narrow, validated `LlmDecisionDraft` — decision, reason, confidence, whether it thinks human review is required. Deliberately smaller than the final `AgentDecision` the system records, because the model's opinion is an input to the guard, not the audit trail. `guards.enforce` takes the *more conservative* of the model's draft and the deterministic policy evaluation — the model may escalate a decision the policy engine would have approved; it may never de-escalate one the policy engine wanted to send to a human. The API key is read from `GROQ_API_KEY` at runtime and is never

hardcoded, logged, or returned by any endpoint; without it, the backend still runs and falls back to the deterministic policy engine alone, marked `decided_by="policy_engine_fallback"`.



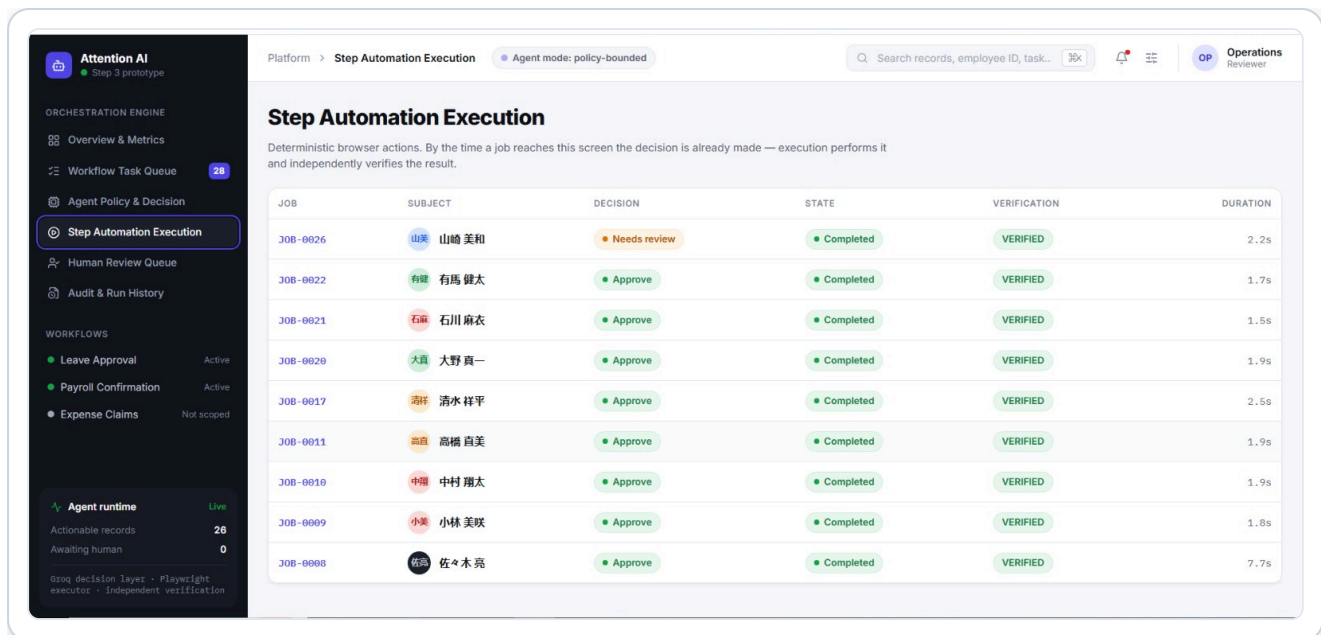
**Figure 16. Agent Policy & Decision** — live decisions across both workflows, with the model's confidence shown per record. Note the mix of **Needs review** (40–60% confidence, policy-reference field ambiguous or missing) and **Approve** (100%, policy fully resolved) — the split is a property of the evidence per record, not a fixed rate.

## 6.6 • The mock internal HR application

A separate, server-rendered web app the platform does not own — the executor's actual target. Both screens mirror what §5.4's visual audit directly observed: field names, the policy-reference note, and both decision buttons per screen (登録確定/保留 for payroll, 承認/差戻し for leave). Every interactive element carries a `data-testid` — the executor's entire contract with the page, never a coordinate or a translatable label.

## 6.7 • Deterministic execution

Playwright performs the decided action — opening the record, clicking the resolved control, entering an optional comment — using the same one browser driver for both workflows, configured per-call rather than at import time. A process-wide lock serialises execution against the shared mock HR system, because concurrent runs would interleave clicks and make verification meaningless.



**Figure 17. Step Automation Execution** — the nine completed runs of that earlier session, every one independently **VERIFIED**. By the time a job reaches this screen the decision is already made; execution performs it and re-checks the result on its own. Every number in this report's text comes from `prototype_metrics.json` (26 runs, 10 executions).

## 6.8 • Verification

"Clicking successfully" and "the record actually changed" are kept as separate fields. Verification independently re-reads the record's status from the *queue* screen — not the detail page that just submitted the action — and compares it against the expected post-action label, resolved before the re-read so the check cannot be biased by what it finds. A run that clicked successfully but produced no state change is reported **FAILED**, not **COMPLETED**.

### A NAMED LIMITATION, NOT A HIDDEN ONE

The post-action status labels (承認済み / 差戻し) were **never observed** in any inspected Dataset B segment — only the button labels were. They are a stated prototype assumption, live in the workflow definition as configurable values, and must be confirmed against whatever system a real deployment targets.

## 6.9 • Human-in-the-loop

Three separate mechanisms route a record to a person rather than letting it execute, and any one of them is sufficient on its own: (1) a three-valued policy check where *unknown* escalates rather than defaulting to pass; (2) the guard's conservatism rule, which can raise a model's approval to review but never lower a review to approve; (3) both negative actions (差戻し, 保留) are configured non-auto-executable, because neither was ever observed being used — there is no evidence of what triggers them or what they obligate downstream. A person's determination is recorded alongside the agent's original decision, never overwriting it, so the audit trail keeps exactly the cases where a human and the policy layer disagreed.



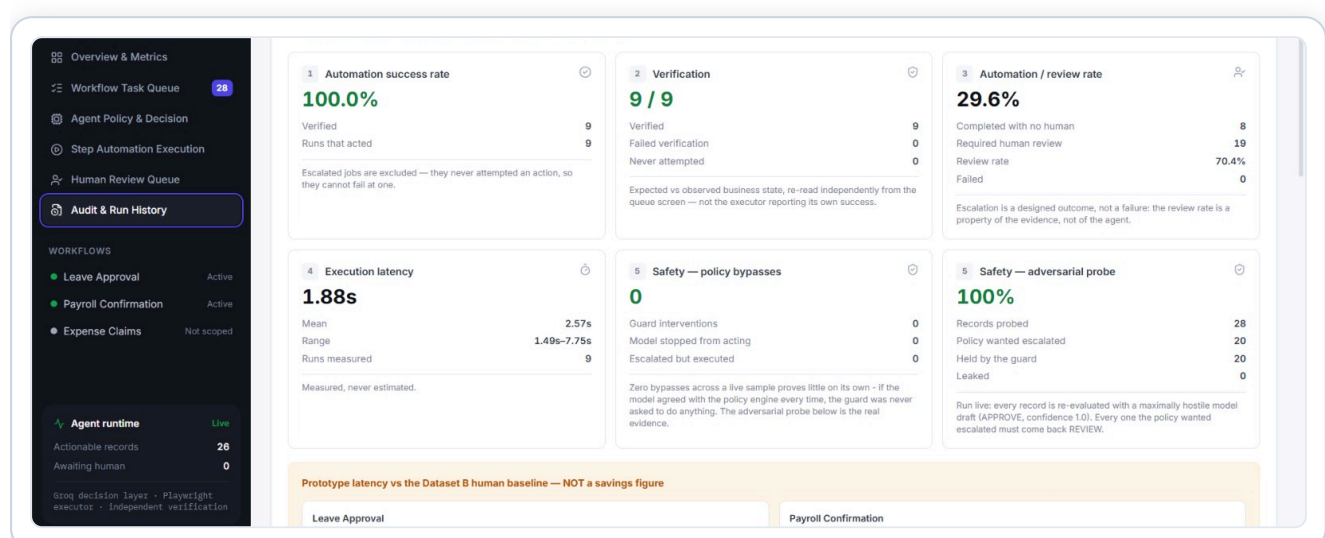
**Figure 18. The decision flow in shape, not in exact module names.** Read *Policy Guardrails* as both the deterministic `policy_engine.py` gates and `guards.enforce` together — two separate boxes in the actual implementation (§6.5–6.8), merged here for shape; “Memory” is illustrative, not a built component (§9 roadmap item 6). The REVIEW / APPROVE branch and the human-review loop-back are drawn exactly as built.

## 7

### SECTION 7

## Phase 3 Evaluation

Every figure below is computed in Python from runs this backend actually performed, at report generation time — the console and this report read the same `/api/metrics` endpoint, so they cannot quietly disagree. Sample: **26 runs, 10 reached the browser** — a demo sample, reported as a count, not dressed up as statistically significant.



**Figure 19. Prototype run metrics**, live from the Audit & Run History screen. The panel is deliberately allowed to say “no data” rather than 0% when nothing has run — the two are different claims. Captured from an earlier demo session than the one §7 measures — the screen’s own counts differ from §7’s; every number in this report’s text comes from `prototype_metrics.json` (26 runs, 10 executions).

### 7.1 • Automation success rate

**10/10 = 100%** of runs that reached the browser were independently verified. Escalated jobs are excluded from this rate by design — they never attempted an action, so counting them would understate a rate that is specifically about execution.

### 7.2 • Verification success

**10/10** verified, 0 failed, 0 never attempted. Verification is the independent re-read described in §6.8, not the executor reporting its own success — the two are structurally different fields, and only the re-read counts here.

### 7.3 • Execution latency

Prototype execution latency: median **1.32s**, mean 1.35s, range 1.26s–1.69s across 10 measured runs. Measured, never estimated.

#### NEVER SUBTRACTED FROM THE HUMAN BASELINE — DIFFERENT POPULATIONS

Prototype latency is browser automation time against a *local mock*; the Dataset B figures are *human* time in a recorded test environment whose waits were deliberately shortened (§3). They are shown side by side purely for the impact discussion in §8, and are never subtracted to produce a savings figure — doing so would compare two populations that were never measured on the same basis.

### 7.4 • Safety / guard evaluation

#### POLICY BYPASSES

**0**

of 26 live runs

#### ADVERSARIAL PROBE HOLD RATE

**100%**

20/20 held, 0 leaked

#### RECORDS PROBED

**28**

every demo record, both workflows

Zero bypasses on a live sample proves little by itself — if the model agreed with the policy engine every time, the guard was never actually tested. The adversarial probe is the real evidence: every one of 28 demo records is re-evaluated with a maximally hostile model draft (APPROVE, confidence 1.0, human review flagged unnecessary), and every one the deterministic policy wanted escalated must come back REVIEW. Run live, at report time, with no LLM and no browser involved — a structural guarantee, not a sampled statistic. Result: **20 of 20 held, 0 leaked**. Claim validated: *"The LLM cannot override deterministic safety controls."*

## 7.5 • Per-workflow findings

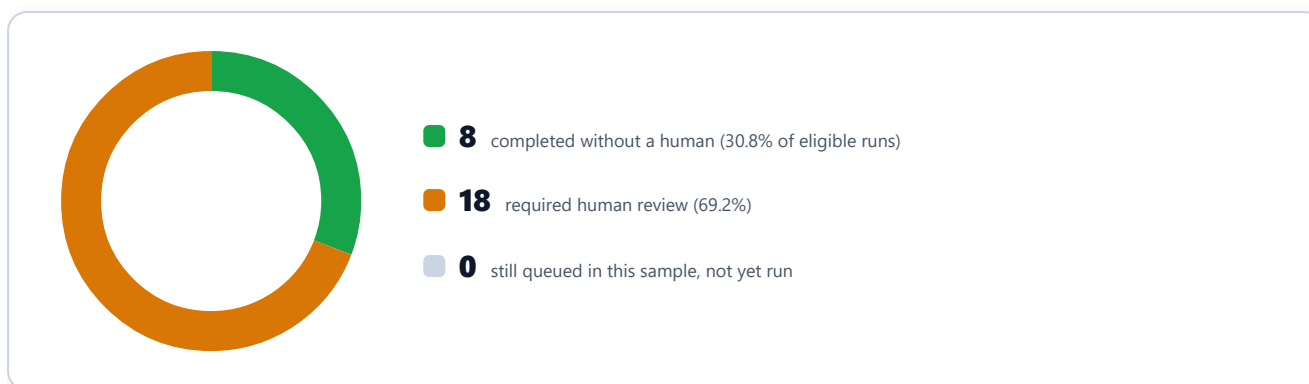


Figure 20. Outcome of every eligible run in the measured sample (26 records). Escalation is the largest slice by design (§8.1), not by shortfall.

| WORKFLOW             | RUNS | REACHED BROWSER | VERIFIED | AUTOMATION RATE | PROTOTYPE MEDIAN |
|----------------------|------|-----------------|----------|-----------------|------------------|
| Leave Approval       | 16   | 5               | 5        | 25%             | 1.34s            |
| Payroll Confirmation | 10   | 5               | 5        | 40%             | 1.30s            |

Payroll automates markedly more than leave (40% vs. 25%) — the reason is specific and evidenced, not incidental, and is the lead finding of §8.

## 8

### SECTION 8

#### BRIEF REQUIREMENT

## Business Impact / ROI Interpretation

### 8.1 • What manual work remains

**69.2% of all eligible runs required human review** in the measured sample (18 of 26). This is stated as the headline number, not a footnote, because escalation here is a *designed* outcome, not a shortfall: a record whose policy gate could not be evaluated — a missing field, an unrecognised category, an unresolved approval-required flag — is supposed to reach a person. The review rate is a property of what the evidence in Dataset B actually revealed, not a property of how capable the agent is.

Concretely, a person is still required for: any record with a missing or unrecognised field; any leave request whose “prior approval required” flag cannot be resolved to *obtained* or *not obtained* (Dataset B showed only whether approval was *required*, never whether it was *obtained* — a genuinely unknown fact, not a guessable one); any payroll item without a readable 参照 policy-reference note; every record below the 0.8 model-confidence floor; and, structurally, every hold (保留) or reject (差戻し) case, because neither action's trigger condition was ever observed.

## 8.2 • Realistic impact — and what is not claimed

### THE HONEST FINDING, STATED ONCE AND REUSED THROUGHOUT THIS SECTION

**Automatability here is bounded by which fields the operator's screen happened to expose, not by how difficult the underlying task is.**

Payroll automates more (40%) than leave (25%) because the payroll policy-reference note was readable on the one record whose detail panel an operator actually opened, while leave's `prior_approval_obtained` was never visible for any record in the sample. Neither task is intrinsically harder than the other — the difference is entirely which piece of evidence the logs happened to capture.

What this measured sample supports: for the specific, narrow policy configured in this prototype, roughly 3 in 10 records of these two types can be processed without a human touching them, verified independently, with zero observed safety-control bypasses. What it does not support, and what this report is deliberately silent on: a production time-savings figure, a headcount or cost-reduction estimate, or an extrapolation from the 226 recovered segments to a production case volume.

- **No savings figure is stated anywhere in this report.** Prototype latency and the Dataset B human baseline are measured on incompatible bases (§7.3) and are never subtracted.
- **No extrapolation from segment counts to production volume.** Only 30.5% of Dataset B's 226 segments are single-app, single-route clean executions (§9); the rest merge multiple business actions, so segment counts are a conservative lower bound on true case volume, and the direction of any error is unknown in scale, only in sign.
- **No claim that the observed 40%/25% split generalises** beyond this specific, hand-authored, demonstration policy. A different, real client policy with different field visibility would very plausibly change the ratio in either direction.

## 9

### SECTION 9 BRIEF REQUIREMENT

## Risks & Mitigations

This project generated an unusually well-documented set of risks — each one found by directly inspecting evidence rather than assumed in the abstract. Each is paired here with a concrete mitigation. Several mitigations are not proposals: they are behaviour already built and verified by the 76-test suite, named here as mitigations rather than left implicit in code.

#### RISK 1

Under-segmentation: 69.5% of segments merge multiple business actions 5132 payroll-items' 39 segments contain 96 distinct case IDs across only 9 of them; the longest onboarding segment (359s) holds at least 14–16 real actions.

#### MITIGATION

The prototype never automates on segment-level evidence directly — it acts per-record, keyed by the detail panel's 管理ID, so a merged log segment cannot cause a merged action. Roadmap: a case-ID boundary-split rule (§10) targets this at the source.

#### RISK 2

Route/label is not a reliable business-function indicator off port 5132 All six '5133 onboarding' segments visually inspected turned out to be Finance/Logistics/IT content, not HR onboarding (§5.4).

#### MITIGATION

The finding is acted on, not just noted: this candidate is excluded from the priority ranking (Table T7) on direct evidence overriding its mechanical tier. Standing rule adopted: visually spot-check any new candidate before scoping automation against it, especially off port 5132.

**RISK 3**

Post-action status labels (承認済み / 差戻し) were never observed  
Only the button labels were confirmed in Dataset B; the resulting status text is inferred, not seen.

**MITIGATION**

Stated explicitly as a prototype assumption inside the workflow definition itself (not buried in a comment), and implemented as a configurable value, not a hardcoded string — a one-line change once the real system's labels are confirmed.

**RISK 4**

The execution target is a mock HR application, not the real system  
Built from screenshots to match what was observed; a real deployment targets a different, unverified DOM.

**MITIGATION**

The executor's entire contract with the page is a stable `data-testid` selector set, declared as data in one file ( `execution/screens.py` ). Pointing at the real system is a ScreenSpec change, not a rewrite of the decision or verification logic (§10, roadmap item 1).

**RISK 5**

Auto-execution evidence rests on a small number of directly observed records  
One clean instance each for payroll and leave; family-level case-ID/extracted-text coverage is under 25% for both.

**MITIGATION**

The policy's `min_confidence_for_auto_execution` (0.8) and the three-valued policy check (unknown escalates) together mean thin evidence routes to a human by construction, regardless of sample size. The adversarial probe (§7.4) proves the guard holds structurally, independent of how many records have been seen.

**RISK 6**

Neither negative action (差戻し reject, 保留 hold) was ever observed being used  
No trigger condition or downstream obligation for either action is visible anywhere in Dataset B.

**MITIGATION**

Both are hard-configured non-auto-executable ( `allow_auto_rejection: false` , `allow_auto_hold: false` ) — not a soft default, a structural block. Enabling either requires a deliberate config change plus new evidence, not silent inclusion.

**RISK 7**

The policy is hand-authored, not learned from the logs  
Dataset B showed that an action was taken, never the rule that justified it — the real approval matrix is unobserved.

**MITIGATION**

Every threshold in `policies.json` carries its own `_rationale` field explaining exactly what evidence (if any) informed it, and a top-level `_meta` block states outright that this is not the client's real policy. Production rollout requires validating against the client's actual approval matrix before auto-execution is enabled (§10).

**RISK 8**

Regex-matched case IDs on a list screen are noise, not the record key  
The case IDs attached to the payroll clean instance (INV-2026-7998...) are dashboard list-view artifacts, not the real record (P1-07046967-001) being processed.

**MITIGATION**

The executor and policy engine key exclusively off the 管理ID/社員ID pair read from the detail panel — never off regex-extracted case IDs from a list view. This was a design decision from the first implementation, not a retrofit.

## 9.1 • What this report does not prove

Stated once, plainly, rather than left for a careful reader to notice: this report does not prove that the shipped policy matches the client's real approval rules (it is explicitly hand-authored, §9 risk 7); does not prove the automation rate generalises beyond this specific demonstration policy and record sample (§8.2); does not prove production time savings (§7.3, §8.2); and does not prove the mock HR application's DOM matches the real target system's DOM (§9 risk 4). What it does prove, structurally and by live re-evaluation rather than by claim: that the deterministic guard cannot be overridden upward by the model (20/20 held under a maximally hostile adversarial draft, §7.4), and that every executed action was independently confirmed to have actually happened (10/10 verified, §7.2).

## 10

## SECTION 10

## Productionization Roadmap

Ordered by what has to happen before the next thing can, not by size of effort.

1. **Point the executor at the real HR system.** The mock exists purely because the real system is not reachable from this environment. Because the executor's contract is a declared `ScreenSpec`, not embedded logic, this is a data change: confirm the real system's `data-testid`s (or add them, coordinating with whoever owns that system) and the real post-action status labels (§9 risk 3), then swap the base URL.
2. **Validate the policy against the client's real approval matrix.** Every threshold currently in `policies.json` is a demonstration value with a stated rationale, not a discovered rule (§9 risk 7). This is a business conversation, not an engineering task, and it gates enabling auto-execution in production.
3. **A case-ID segment-split rule for Phase 1/2, to address under-segmentation at the source.** The dominant remaining error across both phases is merged multi-case sessions, not scorer precision (§4.5, §9 risk 1) — a deterministic rule that splits a segment at a case-ID change (available for a meaningful share of segments per `case_id_presence_fraction`) would directly target this, at the cost of re-validating every downstream count. Deliberately not built for this submission: it would cascade into Phase 2's counts, the backend's evidence figures, and every number in this report, for a benefit that could not be verified without redoing that validation.
4. **Extend to onboarding** (§5.5 rank 2) once a clean, unmerged execution is confirmable — plausibly downstream of roadmap item 3.
5. **Add the negative-action paths** (差戻し, 保留) once real trigger evidence exists for either — today, neither has ever been observed being used (§9 risk 6).
6. **Persist jobs to a database.** The audit trail is currently in-memory and is cleared on restart — acceptable for a demo, not for production.
7. **Reassess the payroll scope across ports.** §5.3 found the same route recurring at three ports; if 5133/5134 are confirmed to be the identical workflow on different instances, the addressable payroll volume is 61 executions (~41 observed minutes), not the 39 this prototype currently covers.

## 11

## SECTION 11

## Conclusion

Three steps, each depending on evidence the one before it produced, not on inference or a clean-looking end state. Phase 1 measured a segmentation approach against ground truth and reported its honest ceiling — how many boundaries carry no detectable signal at all — rather than optimising a metric against an arbitrary bar. Phase 2 applied that frozen approach to production data with no ground truth, found 16 real automation candidates, and then directly inspected the evidence behind the top three before trusting any of it, catching a mislabelled candidate that a purely mechanical ranking would have missed. Phase 3 built two of those candidates end to end, on a pipeline where a deterministic guard structurally cannot be overruled

upward by the model it supervises — demonstrated live, not merely asserted, by an adversarial probe that held every one of 20 records the policy engine wanted escalated.

The honest finding this report leads with is not a percentage: it is that automatability was bounded by which fields an operator's screen happened to expose, not by how difficult either task actually was (§8.2). That finding only exists because the visual audit in §5.4 checked screenshots directly rather than trusting a mechanical label, and because the prototype's every screen distinguishes what was observed from what was fabricated to exercise a code path (§6.4). Every number in this report traces to a committed file (Appendix A) and every reproducible pipeline stage was rerun and checksummed against its own prior output (Appendix B) before being written down here.

Full day-by-day account of how the seven-day budget was actually spent, including what did not work, is recorded in [deliverables/WORKLOG.md](#) rather than repeated here.

A

APPENDIX

Traceability

Every figure and table in this report traces to a specific committed file. This appendix is the index — not because the claims above are in doubt, but because a report whose numbers cannot be checked is exactly the failure mode this entire project has tried to avoid.

| CLAIM                                                          | SOURCE                                                                                                        |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| 226 segments, 15/15 sessions, 15 labels                        | <a href="#">deliverables/segments.jsonl</a>                                                                   |
| Dataset A: 63 sessions / 162,768 events / 2,009 GT executions  | <a href="#">outputs/tables/dataset_a_events.parquet</a> , <a href="#">dataset_a_gt_executions.parquet</a>     |
| Dataset B: 15 sessions / 20,477 events / 4 workers             | <a href="#">outputs/tables/dataset_b_events.parquet</a> , <a href="#">phase2/results/phase2_summary.md §0</a> |
| Gap-threshold baseline scores                                  | <a href="#">phase1/results/phase1_initial/baseline_results.csv</a>                                            |
| 1F–1F.5 tolerance comparison                                   | <a href="#">phase1/results/phase1f5_segmentation/tolerance_comparison.csv</a>                                 |
| 1F.5 held-out precision/recall/F1                              | <a href="#">phase1/results/phase1f5_segmentation/segmentation_statistics.csv</a>                              |
| 16 automation candidates + 5 qualitative tiers                 | <a href="#">phase2/results/automation_candidates.csv</a>                                                      |
| Visual audit of 13 segments, incl. the 5133 onboarding finding | <a href="#">phase2/results/visual_audit.md</a>                                                                |
| 17-point workflow specification, both workflows                | <a href="#">phase2/results/step3_prototype_spec.md</a>                                                        |
| Prototype policy + every threshold's rationale                 | <a href="#">backend/data/policies.json</a>                                                                    |
| Five prototype metrics, this report's §7 figures               | <a href="#">prototype_metrics.json</a> (GET /api/metrics)                                                     |
| Backend architecture, endpoints, key invariants                | <a href="#">backend/README.md</a>                                                                             |
| 76 backend tests, incl. real-browser runs                      | <a href="#">backend/tests/</a> – run via <code>pytest</code> from <code>backend/</code>                       |
| 39 frontend tests                                              | <a href="#">frontend/src/__tests__/</a> – run via <code>npm test</code>                                       |
| Day-by-day work log, incl. AI usage                            | <a href="#">deliverables/WORKLOG.md</a>                                                                       |

## B

## APPENDIX

## Reproducibility &amp; Verification

Nothing in Phase 1 or Phase 2 is random. The scorer is a logistic regression with a fixed `random_state` and fixed `GroupKFold` splits; Phase 2's grouping is rule-based single-linkage at a fixed similarity threshold. Both were rerun after every structural change to this repository and checksummed against their own prior output.

| OUTPUT                                                  | CHECK                                                                                                                    |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| phase1/results/phase1f_segmentation/segment_results.csv | md5 26d3686f... — unchanged across rerun                                                                                 |
| deliverables/segments.jsonl                             | md5 54efb1b5... — unchanged, 226 segments, both copies identical                                                         |
| phase2 frozen-fit reproduction                          | re-fits 1F.5 from scratch; refuses to proceed unless threshold=0.88 and training OOF F1=0.3404874499818116 match exactly |

#### THE REPRODUCIBILITY GATE IS A REFUSAL, NOT A LOG LINE

`phase2/src/phase2_dataset_b_segmentation.py:verify_frozen_reproduction` does not print a warning if the re-fit disagrees with 1F.5's recorded numbers — it stops the pipeline. This ran clean on the actual execution behind every Phase 2 number in this report: the frozen artifact is genuinely reproduced, not approximated.

### Test suites, at time of writing

| SUITE                                                | COUNT      |
|------------------------------------------------------|------------|
| Backend (pytest, incl. real-browser Playwright runs) | 76 passing |
| Frontend (vitest)                                    | 39 passing |
| Phase 0 (data foundation)                            | 2 passing  |

### Environment

One virtual environment, one `requirements.txt`, at the repository root — versions pinned rather than ranged, because a silent minor-version bump in numpy or scikit-learn is exactly what would break the reproducibility claim above quietly. Secrets ( `GROQ_API_KEY` ) are read from the environment at runtime and are never committed, hardcoded, logged, or returned by any API response.